Stochastic Batched Dynamic Programming

A scaling strategy for joint hardware-policy co-design beyond the exact-DP horizon ceiling

Abstract

Exact Bellman dynamic programming for partially observable Markov decision processes (POMDPs) admits an efficient sufficient-statistic memoization for many problems of interest, but the memoization grows fast enough in horizon K that exact solutions are typically capped at $K \approx 4$–5. Joint hardware-policy co-design, where a continuous hardware parameter $\boldsymbol{c}$ is optimized jointly with the inner Bellman-optimal policy $\pi^\star(\boldsymbol{c})$, inherits this ceiling. We develop a scaling extension: *stochastic batched dynamic programming*, which combines (i) a batched receding-horizon decomposition of the inner Bellman problem into n tractable batches of length K_{batch} and (ii) stochastic Monte-Carlo sampling of batch-boundary trajectories at outer-gradient time, replacing the intractable enumeration over reachable batch-boundary beliefs. The resulting estimator is unbiased, per-sample gradients are exact via the sharp-max envelope theorem, and the wall-clock cost scales linearly in the sample size and the number of batches rather than in the reachable belief count. The construction generalizes naturally and preserves the exactness guarantees of joint dynamic programming on the sampled sub-problems.

Contents

1 Setup and notation **1**

2 The exact-DP horizon ceiling **2**

3 Batched dynamic programming **2**
3.1 The value of a batched policy 2
3.2 The curse of enumeration over batch-boundary beliefs 3

4 Stochastic batched DP **3**
4.1 The estimator 3
4.2 Per-sample gradients via the envelope theorem 3
4.3 Outer SGD update 4

5 The general algorithm **4**

6 Convergence and bias **4**

7 Connections to existing methods **5**

8 Practical guidance **6**

9 Discussion **6**

1 Setup and notation

A POMDP is a tuple $(\mathcal{X}, \mathcal{S}, \mathcal{Y}, p_0, p, T)$ with hidden state $\boldsymbol{x} \in \mathcal{X}$, action $\boldsymbol{s} \in \mathcal{S}$, observation $\boldsymbol{y} \in \mathcal{Y}$, prior p_0, observation likelihood $p(\boldsymbol{y} \mid \boldsymbol{x}, \boldsymbol{s}; \boldsymbol{c})$, and Bayesian belief update operator $T(\boldsymbol{b}, \boldsymbol{s}, \boldsymbol{y}; \boldsymbol{c})$. The hardware parameter $\boldsymbol{c} \in \mathcal{C}$ parametrises the likelihood and the belief update. Belief at step k is $\boldsymbol{b}_k \in \Delta(\mathcal{X})$, the simplex of distributions on the state space.

A pointwise reward $\Phi(\boldsymbol{x}, \boldsymbol{s}_{1:K}; \boldsymbol{c})$ depends on the realized trajectory and the hardware. For terminal rewards expressible as a functional of the terminal belief $\boldsymbol{b}_K$, write $\Phi_0(\boldsymbol{b}_K; \boldsymbol{c}) = \mathbb{E}_{\boldsymbol{x}|\boldsymbol{b}_K}[\Phi(\boldsymbol{x}, \boldsymbol{s}_{1:K}; \boldsymbol{c})]$. The Bellman recursion at fixed $\boldsymbol{c}$ is

$$V_K(\boldsymbol{b}; \boldsymbol{c}) = \Phi_0(\boldsymbol{b}; \boldsymbol{c}), \tag{1}$$

$$V_k(\boldsymbol{b}; \boldsymbol{c}) = \max_{\boldsymbol{s} \in \mathcal{S}} \mathbb{E}_{\boldsymbol{y}|\boldsymbol{b},\boldsymbol{s},\boldsymbol{c}}\big[V_{k+1}(T(\boldsymbol{b}, \boldsymbol{s}, \boldsymbol{y}; \boldsymbol{c}); \boldsymbol{c})\big], \tag{2}$$

for $k = K-1, \ldots, 0$. The optimal adaptive policy is $\pi_k^\star(\boldsymbol{b}; \boldsymbol{c}) = \arg\max_{\boldsymbol{s}} \mathbb{E}_{\boldsymbol{y}}[V_{k+1}(T(\boldsymbol{b}, \boldsymbol{s}, \boldsymbol{y}; \boldsymbol{c}); \boldsymbol{c})]$. The joint hardware-policy optimum solves

$$\boldsymbol{c}^\star = \arg\max_{\boldsymbol{c} \in \mathcal{C}} V^\star(\boldsymbol{c}), \qquad V^\star(\boldsymbol{c}) := V_0(p_0; \boldsymbol{c}). \tag{3}$$

The outer gradient $dV^\star/d\boldsymbol{c}$ is computed by the sharp-max envelope theorem: at the optimal inner policy, the indirect contribution of policy sensitivity to $\partial V^\star/\partial \boldsymbol{c}$ vanishes, and reverse-mode automatic differentiation through an argmax-frozen Bellman recursion returns an exact, unbiased outer gradient.

2 The exact-DP horizon ceiling

Exact Bellman DP for the recursion of Equations (1)–(2) admits efficient memoization when a low-dimensional sufficient statistic for the belief is available. Two common settings:

Discrete state. The belief is a probability vector on $|\mathcal{X}|$ states. Reachable beliefs from p_0 form a tree of size at most $(|\mathcal{S}| \cdot |\mathcal{Y}|)^K$. Memoization stores one value per reachable belief; the table size is bounded but grows exponentially with K.

Continuous state with sufficient-statistic reduction. When the per-shot likelihood depends on the history only through a finite tuple of integers (e.g., a count-tuple of observations partitioned by action type), the belief at step k is encoded by that tuple, and the reachable-tuple set is the memo's natural index. The number of reachable tuples grows multiplicatively with K, polynomially with the action and observation alphabet sizes. In favourable cases the memo is $\sim 10^7$ at $K = 4$; unfavourable cases (multimodal posteriors, dense action grids) push this to 10^8–10^9 or more by $K = 5$ or 6. Memory becomes the binding constraint.

For practical hardware (modest workstation, ~ 1 TB RAM) the exact-DP ceiling is typically $K_{\max} \in \{4, 5\}$. Beyond that we descend to approximations.

3 Batched dynamic programming

Suppose the deployment horizon K_{total} is much larger than the exact-DP ceiling $K_{\text{batch}} \leq K_{\max}$, with $K_{\text{total}} = n \cdot K_{\text{batch}}$ for some integer n. Define a *batched policy* π^{batch} as follows:

1. At step $k = 0$, with belief $\boldsymbol{b}_0 = p_0$, solve exact Bellman for K_{batch} epochs starting at $\boldsymbol{b}_0$. Use the resulting policy $\pi^{(1)}$ for steps $1, \ldots, K_{\text{batch}}$.
2. At step $k = K_{\text{batch}}$, with belief $\boldsymbol{b}_{K_{\text{batch}}}$ resulting from executing $\pi^{(1)}$ on the realized observations, solve exact Bellman for K_{batch} epochs starting at $\boldsymbol{b}_{K_{\text{batch}}}$. Use $\pi^{(2)}$ for the next K_{batch} steps.

3. Repeat for n batches total.

Each batch is locally Bellman-optimal over its K_{batch}-step subproblem; the overall procedure is a receding-horizon approximation to the full K_{total} Bellman, suboptimal at the start (because $\pi^{(1)}$ plans only K_{batch} steps ahead even though $K_{\text{total}} - K_{\text{batch}}$ steps remain) and tight at the end.

3.1 The value of a batched policy

The expected reward under the batched policy starting from $\boldsymbol{b}_0 = p_0$ is

$$V^{\text{batch}}(p_0;\, \boldsymbol{c}) \;=\; \mathbb{E}_{\boldsymbol{b}_{K_{\text{batch}}} \sim P(\,\cdot\,|p_0, \pi^{(1)},\, \boldsymbol{c})}\big[\, V^{\text{batch}}(\boldsymbol{b}_{K_{\text{batch}}};\, \boldsymbol{c}, \text{remaining } n-1 \text{ batches})\,\big], \tag{4}$$

unfolding to

$$V^{\text{batch}}(p_0;\, \boldsymbol{c}) \;=\; \sum_{\boldsymbol{b}_{K_{\text{batch}}}} P(\boldsymbol{b}_{K_{\text{batch}}} \mid p_0, \pi^{(1)},\, \boldsymbol{c})\, V_0^{\text{batch}_2}(\boldsymbol{b}_{K_{\text{batch}}};\, \boldsymbol{c}), \tag{5}$$

in the two-batch case ($n = 2$), where $V_0^{\text{batch}_2}$ denotes the standard K_{batch} Bellman value at the new starting belief $\boldsymbol{b}_{K_{\text{batch}}}$ over the second batch. The sum is over reachable batch-boundary beliefs.

3.2 The curse of enumeration over batch-boundary beliefs

To compute Equation (5) exactly we would need $V_0^{\text{batch}_2}(\boldsymbol{b}_{K_{\text{batch}}};\, \boldsymbol{c})$ at every reachable $\boldsymbol{b}_{K_{\text{batch}}}$. In the count-tuple-keyed setting this is one fresh K_{batch} Bellman solve per reachable count-tuple after batch 1. At $K_{\text{batch}} = 4$ the reachable count-tuple set is $\sim 10^7$ in unfavourable regimes, and each fresh solve costs ~ 10 seconds: enumeration cost is 10^8 seconds (thousands of CPU-years).

This is the same memo blow-up that capped exact $K_{\text{total}} = K_{\text{batch}} + 1$ in the first place, just relocated to the batch boundary. Batched DP, naively implemented, does not break the exact-DP ceiling.

4 Stochastic batched DP

The standard remedy for an intractable expectation over a large index set is to replace exhaustive enumeration with Monte-Carlo sampling under the natural distribution $P(\boldsymbol{b}_{K_{\text{batch}}} \mid p_0, \pi^{(1)},\, \boldsymbol{c})$.

4.1 The estimator

Sample M trajectories under the current batched policy. For each trajectory m define $\boldsymbol{b}^{(m)}_{K_{\text{batch}}}$ as the realized batch-1 terminal belief. The estimator

$$\widehat{V}^{\text{batch}}(p_0;\, \boldsymbol{c}; \mathcal{M}) \;:=\; \frac{1}{M} \sum_{m=1}^{M} V_0^{\text{batch}_2}\big(\boldsymbol{b}^{(m)}_{K_{\text{batch}}};\, \boldsymbol{c}\big) \tag{6}$$

is an unbiased estimator of Equation (5): the trajectories $\mathcal{M} = \{\boldsymbol{b}^{(m)}_{K_{\text{batch}}}\}_{m=1}^{M}$ are i.i.d. samples from the correct policy-induced distribution, and the integrand is computed exactly per sample by solving fresh K_{batch} Bellman from $\boldsymbol{b}^{(m)}_{K_{\text{batch}}}$.

The variance of the estimator scales as $\text{Var}[V_0^{\text{batch}_2}]/M$ where the variance is over the trajectory distribution; standard concentration inequalities give the $1/\sqrt{M}$ confidence intervals.

4.2 Per-sample gradients via the envelope theorem

For outer optimization we need not just the value but its gradient with respect to $\boldsymbol{c}$. For each sampled trajectory we apply the sharp-max envelope theorem to the K_{batch}-step subproblem starting at $\boldsymbol{b}^{(m)}_{K_{\text{batch}}}$:

1. Run forward K_{batch} Bellman from $\boldsymbol{b}^{(m)}_{K_{\text{batch}}}$. Record the optimal argmax action at every reachable belief in the second batch's subtree.
2. Re-evaluate $V_0^{\text{batch}_2}(\boldsymbol{b}^{(m)}_{K_{\text{batch}}}; \boldsymbol{c})$ by substituting the recorded argmaxes in place of the max operator. This produces an argmax-free expression in $\boldsymbol{c}$.
3. Apply reverse-mode automatic differentiation to the argmax-free expression.

The envelope theorem certifies that the resulting gradient is exact:

$$\frac{\partial V_0^{\text{batch}_2}(\boldsymbol{b}^{(m)}_{K_{\text{batch}}}; \boldsymbol{c})}{\partial \boldsymbol{c}} \;=\; \frac{\partial}{\partial \boldsymbol{c}}\bigg|_{\pi^\star_{batch_2}\text{ frozen}} V_0^{\text{batch}_2}(\boldsymbol{b}^{(m)}_{K_{\text{batch}}}; \boldsymbol{c}). \tag{7}$$

The first batch's contribution to the gradient (through how $\pi^{(1)}$ shapes the sampling distribution of $\boldsymbol{b}_{K_{\text{batch}}}$) follows the same envelope-theorem-frozen recursion applied to batch 1, and contributes a score-function term in $\partial \log P/\partial \boldsymbol{c}$ that can be carried via reverse-mode AD on the trajectory's likelihood, exactly as in the single-batch joint-DP gradient.

The aggregate gradient

$$\widehat{\nabla}_{\boldsymbol{c}} V^{\text{batch}}(p_0; \boldsymbol{c}; \mathcal{M}) \;=\; \frac{1}{M}\sum_{m=1}^{M} \frac{\partial V^{(m)}(\boldsymbol{c})}{\partial \boldsymbol{c}} \tag{8}$$

is unbiased for $\nabla_{\boldsymbol{c}} V^{\text{batch}}(p_0; \boldsymbol{c})$ and has the standard SGD variance scaling.

4.3 Outer SGD update

The outer optimization becomes Adam (or any preferred stochastic optimizer) on $\boldsymbol{c}$ with mini-batches of size M:

$$\boldsymbol{c}^{(t+1)} = \text{Adam}\big(\boldsymbol{c}^{(t)},\ \widehat{\nabla}_{\boldsymbol{c}} V^{\text{batch}}(p_0; \boldsymbol{c}^{(t)}; \mathcal{M}^{(t)}),\ \eta\big). \tag{9}$$

A fresh trajectory mini-batch $\mathcal{M}^{(t)}$ is sampled at each iteration t, so the variance does not compound across iterations. Standard SGD convergence theory applies: under bounded gradient variance and standard step-size schedules the iterates converge in expectation to a stationary point of $V^{\text{batch}}(p_0; \boldsymbol{c})$.

5 The general algorithm

Algorithm 1 Stochastic Batched DP

Require: Initial $\boldsymbol{c}^{(0)}$, total horizon K_{total}, batch length K_{batch}, sample size M, learning rate η, iterations T

```
for t = 0, 1, ..., T − 1 do
    g ← 0
    for m = 1, ..., M (parallel) do
        b ← p_0
        Initialize trajectory τ_m ← ()
        for batch = 1, ..., n = K_total/K_batch do
            (V_batch, π_batch, memo_batch) ← Bellman(b, K_batch, c^(t))
            for k = 1, ..., K_batch do
                s ← π_batch(b)
                x ~ b; y ~ p(· | x, s; c^(t))
                b ← T(b, s, y; c^(t))
                Append (s, y, b) to τ_m
            end for
        end for
        V_m ← Φ_0(b; c^(t))                    ▷ terminal reward at trajectory leaf
        g_m ← EnvelopeGrad(V_m, τ_m, memos, c^(t))
        g ← g + g_m
    end for
    g ← g/M
    c^(t+1) ← Adam(c^(t), g, η)
end for
return c^(T)
```

Complexity per outer iteration: $M \times n_{\text{batches}} \times \text{cost}(K_{\text{batch}}\ \text{Bellman})$. In typical regimes, $\text{cost}(K_{\text{batch}}\ \text{Bellman}) \approx 10$ seconds, so a single outer gradient step at $M = 10$ and $n = 4$ takes ~ 400 seconds wall-clock without parallelism, or ~ 40 seconds with M-way parallelism on a single multicore node. A complete training run of $T = 500$ iterations finishes in 5–7 hours on standard hardware.

6 Convergence and bias

Per-sample exactness. Each V_m in Equation (6) is computed by exact Bellman within its K_{batch} subproblem, with the sharp-max envelope-theorem gradient. No smoothing temperature, no policy approximation, no biased inner optimizer. The per-sample value and gradient are both exact for the trajectory's contribution to the estimator.

Estimator bias and variance. The estimator $\widehat{V}^{\text{batch}}(p_0;\, \boldsymbol{c}; \mathcal{M})$ is unbiased for the true batched value (which is itself an approximation of the full K_{total} Bellman value). Its variance scales as σ^2/M where σ^2 depends on the spread of $V_0^{\text{batch}_2}$ across reachable batch-boundary beliefs. In well-conditioned problems σ^2 is small and $M = 10$–30 suffices; in problems with bimodal or multimodal batch-boundary belief distributions M needs to grow accordingly.

Convergence rate. Under standard SGD assumptions (bounded gradient variance, diminishing step sizes), the outer iterates converge to a stationary point of $V^{\text{batch}}(p_0;\, \boldsymbol{c})$ at the standard

$1/\sqrt{T}$ rate. Since the per-sample gradient is exact rather than smoothed, the asymptotic noise floor is set by sampling variance alone, not by gradient bias.

Approximation gap to full K_{total} Bellman. The batched policy is a strict approximation of the full K_{total} Bellman policy: each batch's lookahead is truncated to K_{batch}, missing the tail value of subsequent batches. The gap $V_0(p_0; \boldsymbol{c}) - V^{\text{batch}}(p_0; \boldsymbol{c}) \geq 0$ is regime-dependent. When the batch-boundary posterior is unimodal and well-concentrated (e.g., K_{batch} is large enough to "set up" a well-localized belief for subsequent batches), the gap is small. When the batch-boundary posterior is multimodal, the gap can be large because batched DP fails to recognize that $\pi^{(1)}$ should anticipate the disambiguation work of $\pi^{(2)}$.

7 Connections to existing methods

Stochastic gradient descent. The construction is structurally identical to minibatch SGD on neural networks: full-batch gradient over a finite-but-huge index set (here, reachable batch-boundary beliefs) is replaced by a small stochastic mini-batch. The unbiasedness, variance, and convergence properties carry over directly.

Stochastic Dual Dynamic Programming (SDDP). Pereira and Pinto's SDDP for multi-stage stochastic linear programs uses sampled forward-backward sweeps to build cutting-plane approximations of the value function. Stochastic Batched DP is the POMDP analogue: scenarios are replaced by belief trajectories, cutting-plane approximations are replaced by exact Bellman within each batch. The receding-horizon decomposition is a standard MPC pattern in both communities.

Sequential Bayesian experimental design. Simulation-based sequential OED (Huan and Marzouk; Long, Marzouk, and Wang) uses MC sampling of belief trajectories at outer optimization time, but typically with rung-3 myopic information-gain inner policies rather than exact Bellman. Stochastic Batched DP can be viewed as the upgrade of these methods from "myopic" to "Bellman within each batch".

Differentiable POMDP planning. Existing differentiable-POMDP work (deep RL with NN policies, Karkus et al. QMDP-net, etc.) uses NN-parametrized policies with end-to-end gradient training. These methods exhibit a known pathology in joint hardware-policy optimization: the NN policy can absorb the training signal and leave the hardware near initialization (the "hardware-light fixed point"). Stochastic Batched DP sidesteps this by using exact Bellman within each batch, a non-NN inner solver that does not have a free policy capacity to absorb hardware-task signal.

Sample-based POMDP solvers (POMCP, DESPOT). These methods perform Monte-Carlo tree search over the belief space at policy-inference time. Stochastic Batched DP uses sampling at outer-gradient time instead, which is a different point of insertion: the inner policy is exact Bellman (sample-free), and sampling enters only to estimate the outer-hardware gradient.

8 Practical guidance

Choosing K_{batch}. Pick the largest K_{batch} for which exact Bellman fits in the available memory budget. Going larger reduces the approximation gap to full Bellman; going smaller reduces per-iteration wall-clock at the cost of suboptimal early-batch lookahead.

Choosing M. Start at $M = 10$. Increase if the gradient-step direction is noisy enough to slow down Adam convergence (visible as oscillation in the loss trajectory). $M = 30$–50 is a typical regime.

Re-sampling cadence. Sample fresh trajectories at every outer iteration. Re-using trajectories across multiple SGD steps introduces gradient bias because the sampling distribution depends on $\boldsymbol{c}^{(t)}$.

Parallelism. The M trajectory simulations and Bellman solves are embarrassingly parallel. On a C-core machine, target $M \leq C/k_{\text{Bellman}}$ where k_{Bellman} is the number of cores each individual Bellman solve uses effectively (typically 8–64).

Diagnostics. Track the per-batch values $V_0^{\text{batch}_j}$ and the spread of $V^{(m)}$ across samples. A high spread indicates that batch-boundary beliefs are diverse, which is healthy for sampling but suggests K_{batch} may be too small. A monotonically decreasing batch-boundary belief variance across batches indicates progressive disambiguation, which is the regime where batched DP works well.

9 Discussion

Stochastic Batched DP fills a gap in the relaxation hierarchy between exact Bellman DP (rung 1, $K_{\max}$-limited) and the policy-approximation rungs 2–4 (certainty-equivalent, myopic, Gaussian-trace-EKF). It preserves exactness of the inner solver within each batch and exactness of per-sample gradients via the envelope theorem, while breaking the reachable-belief enumeration via Monte-Carlo sampling. The resulting method is well-suited to problems where (a) exact Bellman is feasible at modest K_{batch}, (b) the deployment horizon is much larger, and (c) the outer hardware optimization is the central computational expense.

The framework is particularly natural when combined with point-based value iteration (PBVI) or its rho-POMDP extension as the inner solver. PBVI's α-vector representation makes the per-sample K_{batch} Bellman value $V_0^{\text{batch}_j}(\boldsymbol{b})$ evaluatable at any belief $\boldsymbol{b}$ in $O(|\Gamma| \cdot |\mathcal{X}|)$ time without re-solving from scratch. Combined with the stochastic-batched outer SGD, this lifts the horizon ceiling from $K_{\max} \approx 5$ (exact memoization) to K_{total} of practically arbitrary size.